

Exercice : Inverser une pose à la main

Consigne

Écrire `Inverser(pose)` à la main : conjugué du quaternion, et position opposée tournée par ce conjugué. Vérifier : appliquer la pose à un point, puis l'inverse au résultat, et afficher l'écart au point de départ. Il doit être nul aux arrondis près.

Rappel

Pour une pose $P = (t, q)$, l'inverse est :

$$P^{-1} = (-R_{q^{-1}} \cdot t, q^{-1})$$

où q^{-1} est le conjugué du quaternion unitaire q , c'est-à-dire $(w, -x, -y, -z)$.

La position opposée est d'abord tournée par le conjugué, puis on applique la rotation inverse. Autrement dit, l'inverse de la pose (t, q) est la pose (t', q') avec :

$$\begin{aligned} q' &= q^{-1} = (w, -x, -y, -z) \\ t' &= -R_{q'} \cdot t \end{aligned}$$

Fonction Inverser

```
// Conjugue d'un quaternion unitaire.
Quat Conjugue(const Quat& q)
{
    return { q.w, -q.x, -q.y, -q.z };
}

// Inverse d'une pose : conjugue du quaternion,
// position opposee tournee par ce conjugue.
Pose Inverser(const Pose& pose)
{
    Quat qInv = Conjugue(pose.orientation);

    // Position opposee, tournee par le conjugue.
    Vec3 tOppose = {
        -pose.position.x,
        -pose.position.y,
        -pose.position.z
    };
    Vec3 tInv = Rotation(qInv, tOppose);

    return { tInv, qInv };
}
```

Programme complet

```
#include <array>
#include <cstdio>
#include <cmath>

struct Vec3 { float x, y, z; };
struct Quat { float w, x, y, z; };
struct Pose { Vec3 position; Quat orientation; };
```

```

Vec3 Rotation(const Quat& q, const Vec3& v)
{
    Vec3 u = { q.x, q.y, q.z };
    Vec3 uv = {
        u.y * v.z - u.z * v.y,
        u.z * v.x - u.x * v.z,
        u.x * v.y - u.y * v.x
    };
    Vec3 t = {
        u.y * uv.z - u.z * uv.y + q.w * v.x,
        u.z * uv.x - u.x * uv.z + q.w * v.y,
        u.x * uv.y - u.y * uv.x + q.w * v.z
    };
    return {
        v.x + 2.0f * t.x,
        v.y + 2.0f * t.y,
        v.z + 2.0f * t.z
    };
}

Vec3 Appliquer(const Pose& pose, const Vec3& p)
{
    Vec3 tourne = Rotation(pose.orientation, p);
    return {
        tourne.x + pose.position.x,
        tourne.y + pose.position.y,
        tourne.z + pose.position.z
    };
}

Quat Conjugue(const Quat& q)
{
    return { q.w, -q.x, -q.y, -q.z };
}

Pose Inverser(const Pose& pose)
{
    Quat qInv = Conjugue(pose.orientation);
    Vec3 tOppose = {
        -pose.position.x,
        -pose.position.y,
        -pose.position.z
    };
    Vec3 tInv = Rotation(qInv, tOppose);
    return { tInv, qInv };
}

int main()
{
    Pose pose;
    Vec3 point;

    std::scanf("%f %f %f", &pose.position.x,
        &pose.position.y,
        &pose.position.z);
    std::scanf("%f %f %f %f", &pose.orientation.w,
        &pose.orientation.x,
        &pose.orientation.y,
        &pose.orientation.z);
}

```

```

    std::scanf("%f %f %f", &point.x, &point.y, &point.z);

    // Application de la pose, puis de l'inverse.
    Vec3 transforme = Appliquer(pose, point);
    Pose inverse = Inverser(pose);
    Vec3 retour = Appliquer(inverse, transforme);

    // Ecart au point de depart.
    Vec3 ecart = {
        retour.x - point.x,
        retour.y - point.y,
        retour.z - point.z
    };

    std::printf("Point de depart      : %.4f %.4f %.4f\n",
                point.x, point.y, point.z);
    std::printf("Apres pose          : %.4f %.4f %.4f\n",
                transforme.x, transforme.y, transforme.z);
    std::printf("Apres inverse       : %.4f %.4f %.4f\n",
                retour.x, retour.y, retour.z);
    std::printf("Ecart au depart    : %.6f %.6f %.6f\n",
                ecart.x, ecart.y, ecart.z);

    return 0;
}

```

Exemple d'exécution

Pour l'entrée :

```

1.0 2.0 3.0
0.7071 0.0 0.7071 0.0
1.0 1.0 1.0

```

Le quaternion est une rotation de 90 autour de l'axe y . La sortie est :

```

Point de depart      : 1.0000 1.0000 1.0000
Apres pose          : 0.0000 3.0000 2.0000
Apres inverse       : 1.0000 1.0000 1.0000
Ecart au depart    : 0.000000 0.000000 0.000000

```

L'écart est nul aux arrondis près, ce qui confirme que l'inverse est correct.

Pourquoi la position opposée est tournée

L'inverse d'une pose n'est pas simplement $(-t, q^{-1})$. Il faut tenir compte du fait que la translation t est exprimée dans le repère du parent, et que la rotation inverse s'applique avant la translation inverse.

Formellement, si $P = (t, q)$ transforme un point p en $q \cdot p + t$, alors l'inverse doit transformer $q \cdot p + t$ en p . On cherche $P^{-1} = (t', q')$ tel que :

$$q' \cdot (q \cdot p + t) + t' = p$$

En développant et en utilisant $q' = q^{-1}$:

$$q^{-1} \cdot q \cdot p + q^{-1} \cdot t + t' = p$$

$$p + q^{-1} \cdot t + t' = p$$

Donc :

$$t' = -q^{-1} \cdot t$$

C'est exactement la formule implémentée : la position opposée $-t$, tournée par le conjugué q^{-1} .

À retenir

L'inverse d'une pose s'écrit à la main : conjugué du quaternion, et position opposée tournée par ce conjugué. Le module préfère cette version analytique à l'inversion générale, car celle-ci peut rendre silencieusement l'identité au lieu d'échouer. Une inversion générale, devant une matrice qu'elle ne sait pas inverser, rend l'identité plutôt que d'échouer : le programme continue, l'image sort, et votre caméra est simplement à l'origine. Aucune erreur, aucun message.